

DNS Introduction and Traffic Analysis

OJE DOMINION MAYOWA

FWSD2511507

Computer and Digital Forensics

MR. Aminu Idris

22th September 2026

LAB OVERVIEW

This lab examined DNS resolution using dig and packet captures to analyze queries, responses, and browser-generated traffic. I decoded query/response fields, matched transaction IDs between requests and answers, calculated response times, and inventoried the multiple background DNS queries generated during a typical browser session.

OBJECTIVES

- Explain recursive and iterative DNS resolution at a foundational level.
- Use dig to query A, AAAA, MX and NS records.
- Capture and analyze DNS queries and responses with tshark/Wireshark.
- Identify transaction ID, query name, record type, response code, answer and TTL.
- Correlate DNS responses with later IP connections.
- Recognize normal variations such as multiple answers, CNAMEs, cache responses and IPv6 queries.

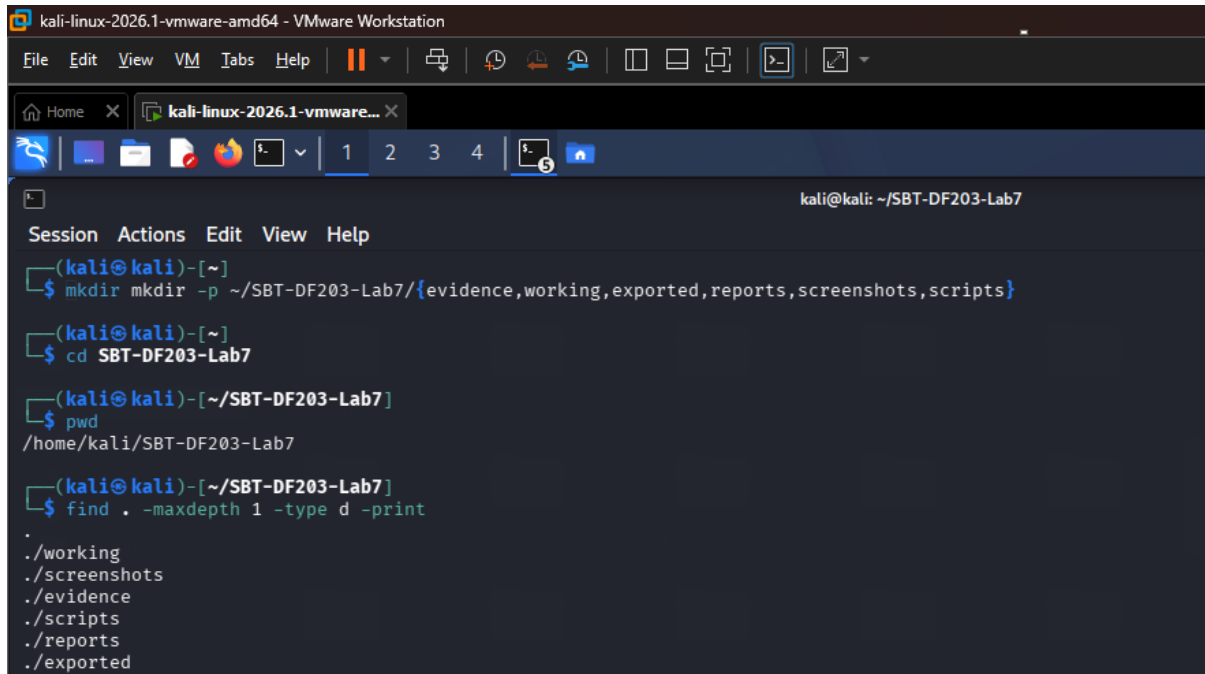
TOOLS

- Authorized training Evidence
- Kali Linux
- Microsoft Word
- NetfilterQueue
- Md5sum and sha256sum
- iptables, iptables-save
- Firefox, curl
- Wireshark, tshark

METHODOLOGY

Lab Folder Structure and Evidence Preparation

First, we create the Lab environment needed for this lab work

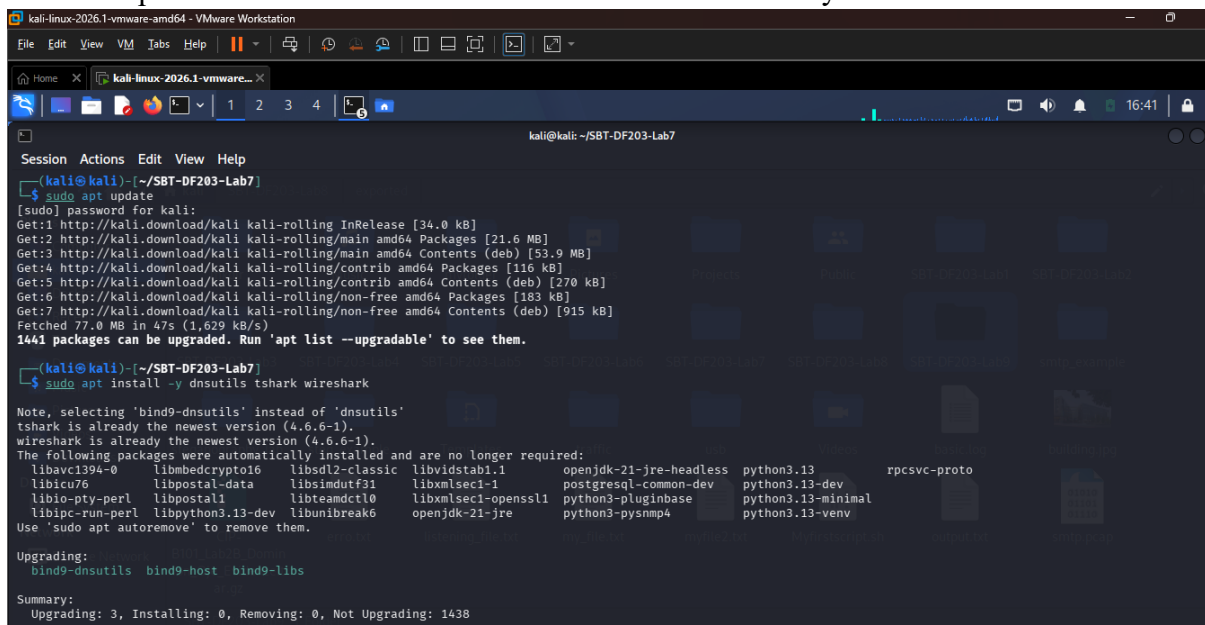


The screenshot shows a Kali Linux terminal window titled "kali-linux-2026.1-vmware-amd64 - VMware Workstation". The terminal displays the following commands and output:

```
kali@kali: ~/SBT-DF203-Lab7
Session Actions Edit View Help
(kali@kali)-[~]
$ mkdir -p ~/SBT-DF203-Lab7/{evidence,working,exported,reports,screenshots,scripts}
(kali@kali)-[~]
$ cd SBT-DF203-Lab7
(kali@kali)-[~/SBT-DF203-Lab7]
$ pwd
/home/kali/SBT-DF203-Lab7
(kali@kali)-[~/SBT-DF203-Lab7]
$ find . -maxdepth 1 -type d -print
.
./working
./screenshots
./evidence
./scripts
./reports
./exported
```

Fig 1: Creating the Lab Environment

Now we update the environment and we download the necessary tools needed for the lab

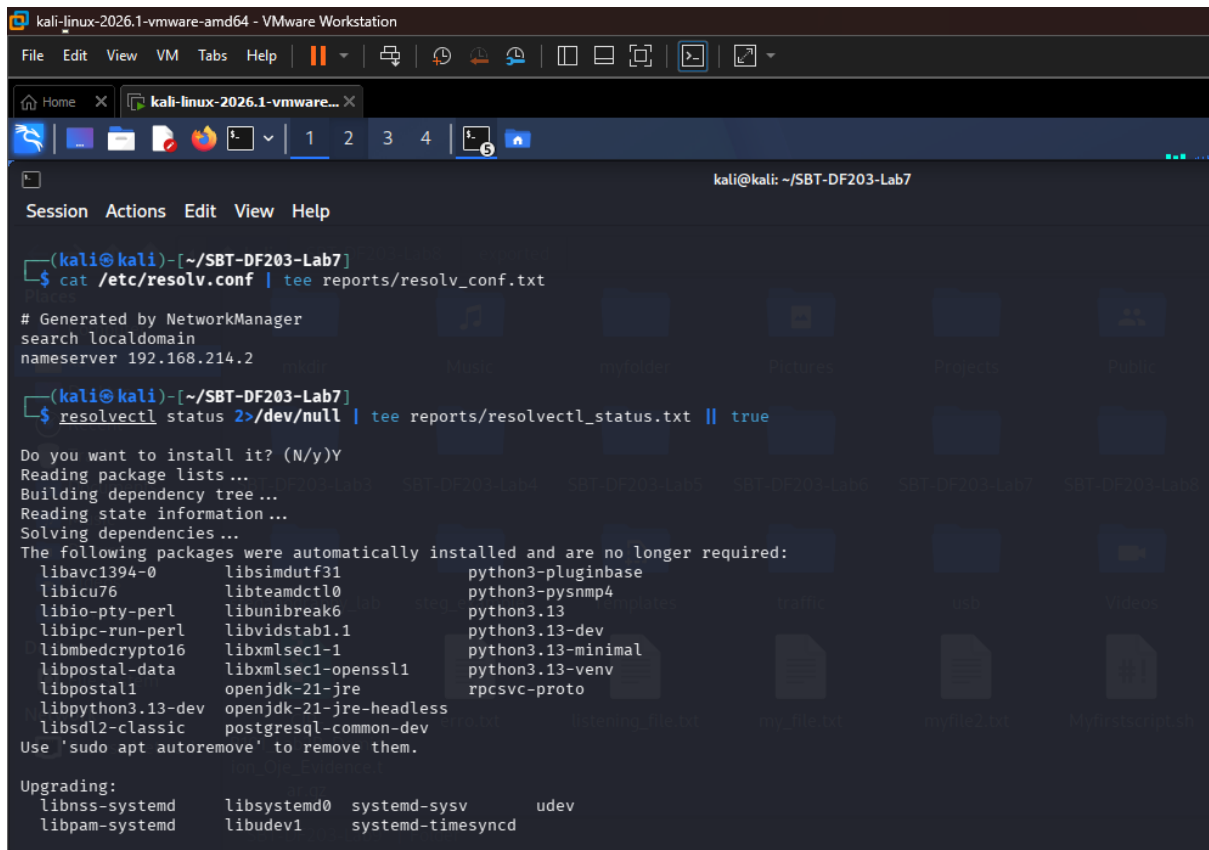


The screenshot shows a Kali Linux terminal window titled "kali-linux-2026.1-vmware-amd64 - VMware Workstation". The terminal displays the following commands and output:

```
kali@kali: ~/SBT-DF203-Lab7
Session Actions Edit View Help
(kali@kali)-[~/SBT-DF203-Lab7]
$ sudo apt update
[sudo] password for kali:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.6 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.9 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [116 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [270 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]
Fetched 77.0 MB in 47s (1,629 kB/s)
1441 packages can be upgraded. Run 'apt list --upgradable' to see them.
(kali@kali)-[~/SBT-DF203-Lab7]
$ sudo apt install -y dnsutils tshark wireshark
Note, selecting 'bind9-dnsutils' instead of 'dnsutils'
tshark is already the newest version (4.6.6-1).
wireshark is already the newest version (4.6.6-1).
The following packages were automatically installed and are no longer required:
  libbav1994-0 libbbedcrypto16 libbsd2-classic libbvidstab1.1 openjdk-21-jre-headless python3.13 rpcsvc-proto
  libicu76 libpostal-data libsimdtf31 libxmlsec1-1 postgresql-common-dev python3.13-dev python3.13-minimal
  libio-pty-perl libpostal1 libteamctl0 libxmlsec1-openssl python3-pluginbase python3.13-venv
  libipc-run-perl libpython3.13-dev libunibreak6 openjdk-21-jre python3-pysnmp4 python3.13-venv
Use 'sudo apt autoremove' to remove them.
Upgrading:
  bind9-dnsutils bind9-host bind9-libs
Summary:
  Upgrading: 3, Installing: 0, Removing: 0, Not Upgrading: 1438
```

Fig 2: Updating environment and downloading necessary tools

Then we get the details of the evidence or the source file we would be working with. Firstly, we check the content of the config file



```
kali@kali: ~/SBT-DF203-Lab7
$ cat /etc/resolv.conf | tee reports/resolv_conf.txt
# Generated by NetworkManager
search localdomain
nameserver 192.168.214.2

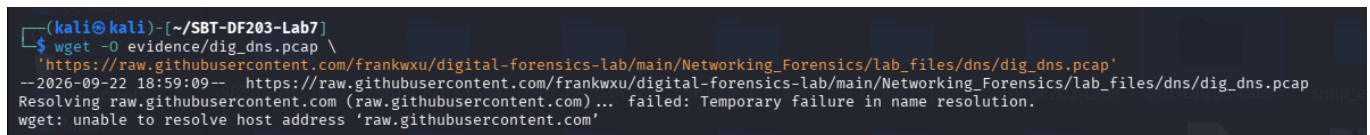
(kali@kali) ~/SBT-DF203-Lab7
$ resolvectl status 2>/dev/null | tee reports/resolvectl_status.txt || true

Do you want to install it? (N/y)Y
Reading package lists...
Building dependency tree...
Reading state information...
Solving dependencies...
The following packages were automatically installed and are no longer required:
libavc1394-0 libsimdtcf31 python3-pluginbase
libicu76 libteamdctl0 python3-pysnmp4
libio-pty-perl libunibreak6 python3.13
libipc-run-perl libvidstab1.1 python3.13-dev
libmbedcrypto16 libxmlsec1-1 python3.13-minimal
libpostal-data libxmlsec1-openssl1 python3.13-venv
libpostal1 openjdk-21-jre rpcsvc-proto
libpython3.13-dev openjdk-21-jre-headless
libstdl2-classic postgresql-common-dev
Use 'sudo apt autoremove' to remove them.

Upgrading:
libnss-systemd libsystemd0 systemd-sysv udev
libpam-systemd libudev1 systemd-timesyncd
```

Fig 3: Getting the Source for the Lab-work

So, in the bid to get the evidence source for the lab work I came across a road block as I couldn't download the evidence source due to a broken DNS server.



```
(kali@kali) ~/SBT-DF203-Lab7
$ wget -O evidence/dig_dns.pcap \
'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns/dig_dns.pcap'
--2026-09-22 18:59:09-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns/dig_dns.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... failed: Temporary failure in name resolution.
wget: unable to resolve host address 'raw.githubusercontent.com'
```

Fig 4: Could not download the Evidence Source

But then I resolved it by fixing the DNS Server or directly contacting the DNS server by using the echo command to put the server in the **resolv.conf** file



```
kali@kali: ~/SBT-DF203-Lab7
$ echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf
nameserver 8.8.8.8

(kali@kali) ~/SBT-DF203-Lab7
$ wget -O evidence/dig_dns.pcap \
'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns/dig_dns.pcap'
--2026-09-22 19:05:05-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns/dig_dns.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.109.133, 185.199.108.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.109.133|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1804656 (1.7M) [application/octet-stream]
Saving to: 'evidence/dig_dns.pcap'

evidence/dig_dns.pcap      100%[=====] 1.72M  3.91MB/s  in 0.4s
2026-09-22 19:05:10 (3.91 MB/s) - 'evidence/dig_dns.pcap' saved [1804656/1804656]
```

Fig 5: Successfully Downloaded the Evidence Source

Then we copied the original copy to preserve the integrity of the original evidence and then we took the hash value of both the original and the working copy.

```
(kali㉿kali)-[~/SBT-DF203-Lab7]
└─$ cp --preserve=timestamps evidence/dig_dns.pcap working/dig_dns_working.pcap

(kali㉿kali)-[~/SBT-DF203-Lab7]
└─$ sha256sum evidence/dig_dns.pcap working/dig_dns_working.pcap | tee reports/dns_capture_hashes.txt
9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d evidence/dig_dns.pcap
9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d working/dig_dns_working.pcap

(kali㉿kali)-[~/SBT-DF203-Lab7]
└─$
```

Fig 6: Take a copy of the original Evidence and the hash value

Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	SBT-DF203-Lab7-Oje Dominion
Trainee name	OJE DOMINION MAYOWA
Date and time started	22/09/2026, 5:00PM
Evidence file name(s)	dig_dns.pcap
Source or generation method	Downloaded from a github link: (https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns/dig_dns.pcap)
Original SHA-256	9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d
Working-copy SHA-256	9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d
Analysis workstation/VM	Kali VM
Notes on any changes	

Part A - Query DNS Records with dig

```
(kali㉿kali)-[~/SBT-DF203-Lab7]
$ dig example.com A | tee reports/dig_example_A.txt

; <<>> DiG 9.20.27-2-Debian <<>> example.com A
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 41220
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;example.com.                IN      A
;; ANSWER SECTION:
example.com.                 300     IN      A       172.66.147.243
example.com.                 300     IN      A       104.20.23.154

;; Query time: 122 msec
;; SERVER: 8.8.8.8#53(8.8.8.8) (UDP)
;; WHEN: Sat Sep 26 10:28:43 WAT 2026
;; MSG SIZE rcvd: 72

(kali㉿kali)-[~/SBT-DF203-Lab7]
$
```

Fig 7: DNS A record query results for example.com

- ☐ **Resolver:** 8.8.8.8 (Google DNS, UDP)
- ☐ **Status:** NOERROR
- ☐ **Flags:** qr rd ra ad
- ☐ **Question count:** 1
- ☐ **Answer count:** 2
- ☐ **Query time:** 122 msec
- ☐ **IN:** Internet class (This is standard for all DNS records)
- ☐ **A:** This maps a domain name to an IPv4 address
- ☐ **TTL:** 300 seconds - cache lifetime, not a creation date

```
(kali@kali)-[~/SBT-DF203-Lab7]
$ dig example.com AAAA | tee reports/dig_example_AAAA.txt

; <<>> DiG 9.20.27-2-Debian <<>> example.com AAAA
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 7534
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;example.com.                IN      AAAA

;; ANSWER SECTION:
example.com.                300     IN      AAAA    2606:4700:10::ac42:93f3
example.com.                300     IN      AAAA    2606:4700:10::6814:179a

;; Query time: 126 msec
;; SERVER: 8.8.8.8#53(8.8.8.8) (UDP)
;; WHEN: Sat Sep 26 10:42:07 WAT 2026
;; MSG SIZE rcvd: 96
```

Fig 8: DNS AAA record query results for example.com

- ☐ **Resolver:** 8.8.8.8 (Google DNS, UDP)
- ☐ **Status:** NOERROR
- ☐ **Flags:** qr rd ra ad
- ☐ **Question count:** 1
- ☐ **Answer count:** 2
- ☐ **Query time:** 126 msec
- ☐ **IN:** Internet class (This is standard for all DNS records)
- ☐ **A:** This maps a domain name to an IPv4 address
- ☐ **TTL:** 300 seconds - cache lifetime, not a creation date

```
(kali㉿kali)-[~/SBT-DF203-Lab7]
$ dig example.com MX | tee reports/dig_example_MX.txt

; <<>> DiG 9.20.27-2-Debian <<>> example.com MX
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 48465
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;example.com.                IN      MX
;; ANSWER SECTION:
example.com.                 56      IN      MX      0 .

;; Query time: 51 msec
;; SERVER: 8.8.8.8#53(8.8.8.8) (UDP)
;; WHEN: Sat Sep 26 11:01:37 WAT 2026
;; MSG SIZE rcvd: 55

(kali㉿kali)-[~/SBT-DF203-Lab7]
$
```

Fig 9: DNS MX record query results for example.com

- ☐ **Resolver:** 8.8.8.8 (Google DNS, UDP)
- ☐ **Status:** NOERROR
- ☐ **Flags:** qr rd ra ad
- ☐ **Question count:** 1
- ☐ **Answer count:** 2
- ☐ **Query time:** 51 msec
- ☐ **IN:** Internet class (This is standard for all DNS records)
- ☐ **A:** This maps a domain name to an IPv4 address
- ☐ **TTL:** 300 seconds - cache lifetime, not a creation date

```
(kali㉿kali)-[~/SBT-DF203-Lab7]
$ dig example.com NS | tee reports/dig_example_NS.txt

; <<>> DiG 9.20.27-2-Debian <<>> example.com NS
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 60839
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;example.com.                IN      NS

;; ANSWER SECTION:
example.com.                21600   IN      NS      hera.ns.cloudflare.com.
example.com.                21600   IN      NS      elliottns.cloudflare.com.

;; Query time: 145 msec
;; SERVER: 8.8.8.8#53(8.8.8.8) (UDP)
;; WHEN: Sat Sep 26 11:22:52 WAT 2026
;; MSG SIZE rcvd: 95

(kali㉿kali)-[~/SBT-DF203-Lab7]
$
```

Fig 10: DNS NS record query results for example.com

- ☐ **Resolver:** 8.8.8.8 (Google DNS, UDP)
- ☐ **Status:** NOERROR
- ☐ **Flags:** qr rd ra ad
- ☐ **Question count:** 1
- ☐ **Answer count:** 2
- ☐ **Query time:** 145 msec
- ☐ **IN:** Internet class (This is standard for all DNS records)
- ☐ **A:** This maps a domain name to an IPv4 address
- ☐ **TTL:** 300 seconds - cache lifetime, not a creation date

```
(kali㉿kali)-[~/SBT-DF203-Lab7]
$ dig +short example.com A | tee reports/dig_example_short.txt
104.20.23.154
172.66.147.243

(kali㉿kali)-[~/SBT-DF203-Lab7]
$
```

Fig 11: DNS A record query results for example.com

Part B - Capture a Fresh DNS Query

Now we want to capture a fresh DNS query.

```
(kali㉿kali)-[~/SBT-DF203-Lab7]
$ IFACE=eth0
tshark -i "$IFACE" -f 'port 53' -a duration:25 -w evidence/fresh_dig_dns.pcapng &
sleep 3
[1] 2706487

(kali㉿kali)-[~/SBT-DF203-Lab7]
$ Capturing on 'eth0'

[1] + done      tshark -i "$IFACE" -f 'port 53' -a duration:25 -w
```

Fig 12: Captured a fresh DNS Query

Then I get the output and also get the hash value

```
(kali㉿kali)-[~/SBT-DF203-Lab7]
$ ls -lh evidence/fresh_dig_dns.pcapng
tshark -r evidence/fresh_dig_dns.pcapng | wc -l
sha256sum evidence/fresh_dig_dns.pcapng | tee reports/fresh_dns_sha256.txt
-rw-r--r-- 1 kali kali 580 Sep 26 12:43 evidence/fresh_dig_dns.pcapng
2
c61b96c77de9932193fac5f453412903d4a2bcd4243a3f726d8c24bb3ee390d1  evidence/fresh_dig_dns.pcapng

(kali㉿kali)-[~/SBT-DF203-Lab7]
$
```

Fig 13: Checking the Integrity of the Captured Query

Part C - Extract DNS Query and Response Fields

0x55b9

```
(kali㉿kali)-[~/SBT-DF203-Lab7]
$ PCAP=evidence/fresh_dig_dns.pcapng
# Query
tshark -r "$PCAP" -Y 'dns.flags.response==0' -T fields \
-e frame.number -e frame.time -e ip.src -e udp.srcport -e ip.dst -e udp.dstport \
-e dns.id -e dns.qry.name -e dns.qry.type \
| tee reports/dns_queries.tsv
1      2026-09-26T12:43:26.578939184+0100      192.168.214.128 41679      8.8.8.8 53      0x55b9  example.com      1
```

Fig 14: The DNS Query Field

```
(kali㉿kali)-[~/SBT-DF203-Lab7]
$ # Response
tshark -r "$PCAP" -Y 'dns.flags.response==1' -T fields \
-e frame.number -e frame.time -e ip.src -e udp.srcport -e ip.dst -e udp.dstport \
-e dns.id -e dns.flags.rcode -e dns.count.answers -e dns.a -e dns.aaaa -e dns.resp.ttl \
| tee reports/dns_responses.tsv
2      2026-09-26T12:43:26.947295683+0100      8.8.8.8 53      192.168.214.128 41679      0x55b9  0      2      172.66.147.243,104.20.23.154      300,300

(kali㉿kali)-[~/SBT-DF203-Lab7]
$
```

Fig 15: The DNS Response Field

Part D - Match Queries to Responses

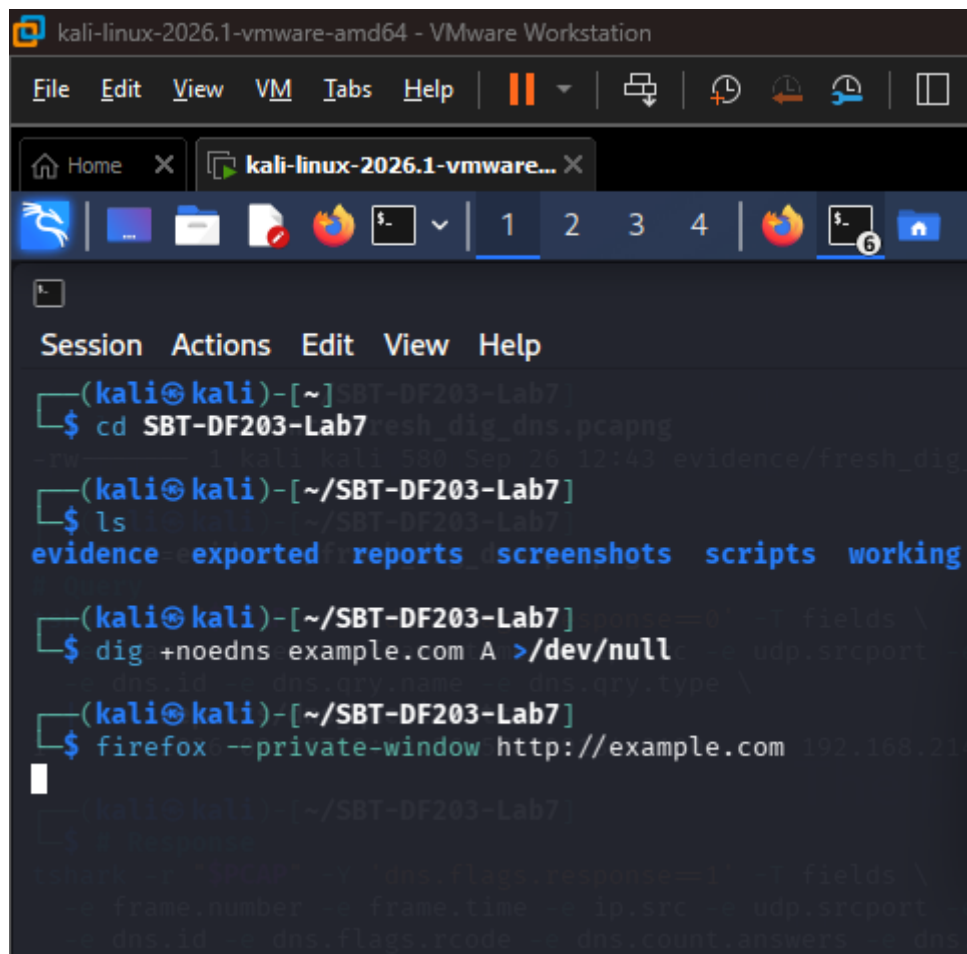
Transaction ID	Query Name	Type	Client/Resolver	Response Code	Answer(s)	TTL	Time Delta
0x55b9	example.com	1	Client: 192.168.214.128:41679 Resolver: 8.8.8.8:53	0 (which means no error)	172.66.147.243, 104.20.23.154	300, 300 (both answers share the same TTL)	368 ms

Part E - Analyze DNS Generated by a Browser

```
(kali㉿kali)-[~/SBT-DF203-Lab7]
└─$ tshark -i any -f 'port 53' -a duration:40 -w evidence/browser_dns.pcapng
Capturing on 'any'
tshark: Promiscuous mode not supported on the "any" device.
38

(kali㉿kali)-[~/SBT-DF203-Lab7]
└─$
```

Fig 16: Opening a Listener to get traffic



```
kali-linux-2026.1-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Home X kali-linux-2026.1-vmware... X
1 2 3 4 5 6
Session Actions Edit View Help
(kali@kali)-[~] SBT-DF203-Lab7
$ cd SBT-DF203-Lab7
$ ls
evidence exported reports screenshots scripts working
(kali@kali)-[~/SBT-DF203-Lab7]
$ dig +noedns example.com A >/dev/null
(kali@kali)-[~/SBT-DF203-Lab7]
$ firefox --private-window http://example.com 192.168.21
(kali@kali)-[~/SBT-DF203-Lab7]
$ # Response
# Check for IP-CAPTIVE dns.flags.response=1 -> 1 fields \
frame.number <- frame.time <- ip.src <- udp.srcport
dns.id <- dns.flags.rcode <- dns.count.answers <- dns
```

Fig 17: Generating DNS Traffic from Browser

This will show all the unique domain the browser queried, sorted by how many times each was requested, and it is useful for spotting the main site alongside any secondary domains (CDNs, fonts, analytics/telemetry) that got pulled in automatically.

```

kali-linux-2026.1-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2026.1-vmware...
kali@kali: ~/SBT-DF203-Lab7
Session Actions Edit View Help
(kali@kali)-[~/SBT-DF203-Lab7]
$ tshark -i any -f 'port 53' -a duration:40 -w evidence/browser_dns.pcapng
Capturing on 'any'
tshark: Promiscuous mode not supported on the "any" device.
38

(kali@kali)-[~/SBT-DF203-Lab7]
$ tshark -r evidence/browser_dns.pcapng -Y 'dns.flags.response==0' -T fields -e dns.qry.name -e dns.qry.type \
| sort | uniq -c | sort -nr | tee reports/browser_dns_inventory.txt
  2 example.org      28
  1 versioncheck-bg.addons.mozilla.org  1
  1 versioncheck-bg.addons.mozilla.org  1
  1 services.addons.mozilla.org      28
  1 services.addons.mozilla.org      1
  1 ipv4only.arpa      28
  1 ipv4only.arpa      1
  1 firefox.settings.services.mozilla.com  28
  1 firefox.settings.services.mozilla.com  1
  1 firefox-settings-attachments.cdn.mozilla.net  28
  1 firefox-settings-attachments.cdn.mozilla.net  1
  1 example.org      28
  1 example.com      65
  1 content-signature-2.cdn.mozilla.net  28
  1 content-signature-2.cdn.mozilla.net  1
  1 cloudflare-dns.com      65
  1 aus5.mozilla.org      28
  1 aus5.mozilla.org      1
(kali@kali)-[~/SBT-DF203-Lab7]
$

```

Fig 18: Unique Domain browser generated

Part F - Correlate DNS with Subsequent Connections

```

(kali@kali)-[~/SBT-DF203-Lab7]
$ tshark -r evidence/browser_dns.pcapng -Y 'dns.a' -T fields -e frame.time_epoch -e dns.qry.name -e dns.a \
| tee reports/dns_A_answers.tsv
1790439091.411289379 content-signature-2.cdn.mozilla.net 34.149.226.178
1790439111.621543844 firefox.settings.services.mozilla.com 151.101.65.91,151.101.129.91,151.101.193.91,151.101.1.91
1790439113.174814368 firefox-settings-attachments.cdn.mozilla.net 151.101.65.91,151.101.129.91,151.101.193.91,151.101.1.91
1790439114.651319200 services.addons.mozilla.org 151.101.193.91,151.101.65.91,151.101.1.91,151.101.129.91
1790439115.438430574 example.org 104.20.26.136,172.66.157.237
1790439115.729424570 example.org 172.66.157.237,104.20.26.136
1790439115.956101274 ipv4only.arpa 192.0.0.170,192.0.0.171
1790439116.919115377 versioncheck-bg.addons.mozilla.org 34.160.90.233
1790439117.046345861 aus5.mozilla.org 151.101.129.91,151.101.1.91,151.101.193.91,151.101.65.91
(kali@kali)-[~/SBT-DF203-Lab7]
$

```

Fig 19: Extracting DNS A Answer

So, while taking the capture of example.com I didn't get any tcp packets so I had to check all the packets that came in such as ip, udp etc. so I had to run an open port to capture the tcp packets

```

(kali@kali)-[~/SBT-DF203-Lab7]
$ tshark -r evidence/browser_dns.pcapng -T fields -e frame.protocols | sort | uniq -c
38 sll:ethertype:ip:udp:dns

(kali@kali)-[~/SBT-DF203-Lab7]
$ tshark -i any -w evidence/browser_full.pcapng
Capturing on 'any'
tshark: Promiscuous mode not supported on the "any" device.
221 ^C

```

Fig 20: Capturing Tcp Packets

```
(kali@kali)-[~/SBT-DF203-Lab7]
$ firefox --private-window http://example.org
```

Fig 21: Generating Traffic

Part G - DNS Analysis Within SMTP Evidence

```
(kali@kali)-[~/SBT-DF203-Lab7]
$ tshark -r evidence/browser_full.pcapng -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
  -e frame.time_epoch -e ip.dst -e tcp.dstport \
  | tee reports/subsequent_tcp_destinations.tsv
```

1790444102.515569217	127.0.0.1	8080
1790444102.537841980	127.0.0.1	8080
1790444103.292597289	127.0.0.1	8080
1790444103.295057958	127.0.0.1	8080
1790444109.211901608	127.0.0.1	8080
1790444109.214922801	127.0.0.1	8080
1790444109.217111034	127.0.0.1	8080
1790444109.218252381	127.0.0.1	8080
1790444109.220393837	127.0.0.1	8080
1790444109.222830815	127.0.0.1	8080
1790444109.223364362	127.0.0.1	8080
1790444109.225527571	127.0.0.1	8080
1790444109.368673184	127.0.0.1	8080
1790444109.375110049	127.0.0.1	8080
1790444109.515974737	127.0.0.1	8080
1790444109.783750828	34.149.226.178	443
1790444109.784446596	34.149.226.178	443
1790444109.786103854	34.149.226.178	443
1790444109.842740467	151.101.129.91	443
1790444109.938019714	127.0.0.1	8080
1790444110.088041309	151.101.129.91	443
1790444111.498764097	127.0.0.1	8080
1790444111.508945461	127.0.0.1	8080

Fig 21: TCP Packets generated

Part G - DNS Analysis Within SMTP Evidence

```
(kali@kali)-[~/SBT-DF203-Lab7]
$ # Optional, adjust path to your Lab 4 capture
tshark -r ../SBT-DF203-Lab4/working/smtp_working.pcap -Y 'dns' -T fields \
  -e frame.number -e frame.time -e dns.qry.name -e dns.qry.type -e dns.a -e dns.resp.name -e dns.resp.ttl \
  | tee reports/smtp_dns_correlation.tsv
```

Frame Number	Time	Query Name	Query Type	IP	Response Name	TTL
1	2009-10-05T07:06:07.492060000+0100	mail.patriots.in	1			
2	2009-10-05T07:06:07.526085000+0100	mail.patriots.in	1	74.53.140.153	mail.patriots.in,patriots.in,patriots.in,patriots.in	10827,10828,828,82828

Fig 22: Adjusting path to Lab 4

Required Findings Worksheet

Field	Finding
Configured resolver IP	8.8.8.8 (Google Public DNS)
Client source port	41679
Resolver destination port	53
Transaction ID	0x55b9
Query name and type	example.com, type A
Response code	0 - NOERROR
Answer IP(s)	172.66.147.243, 104.20.23.154
TTL	300 seconds
Query-response time delta	0.368 seconds (368ms)
Subsequent connection correlation	There was no connection observed the capture was filtered to port 53 only, so the follow-up TCP connection wasn't recorded.
Normal variations observed	Multiple A records for load balancing, consistent TTLs, and several background DNS queries from Firefox itself (updates, telemetry, settings) beyond the main site.